School of Computer Science and Artificial Intelligence

Lab Assignment # 05

Course Title	**: AI Assistant Coding**
Name of Student	**: M. Sai Pallavi**
Enrollment No.	**: 2303A54063**
Batch No.	**: 48**

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #1 (Privacy in API Usage)

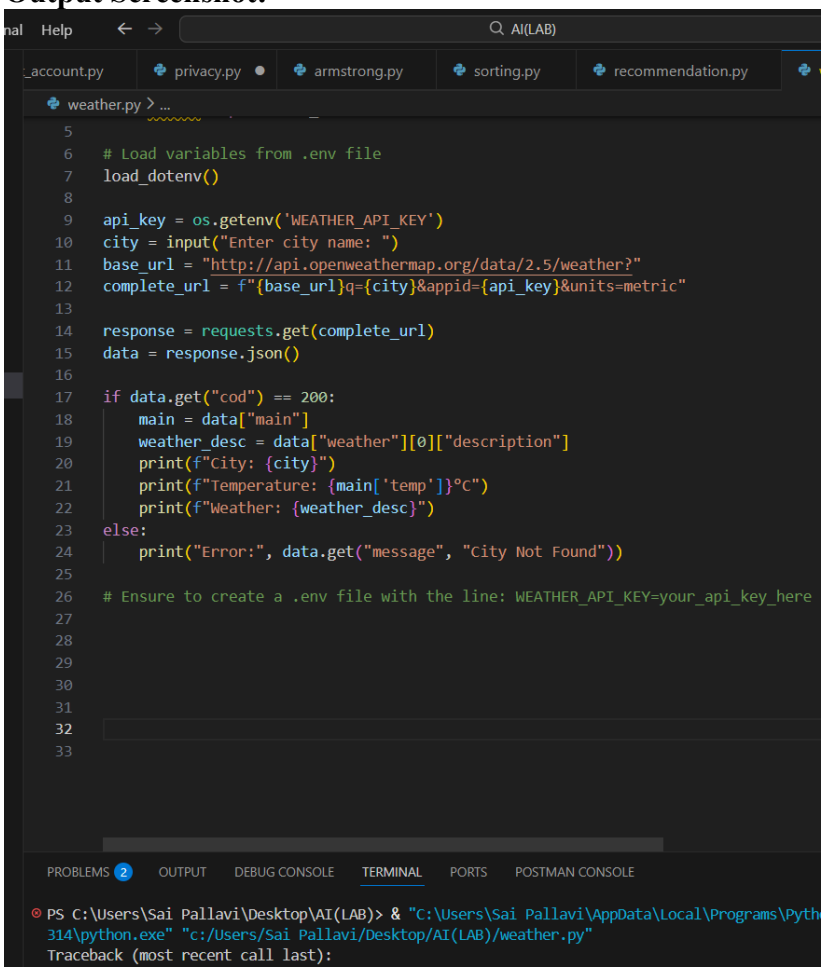
Task: Use an AI tool to generate a Python program that connects to a weather API.

Prompt: "Generate code to fetch weather data securely without exposing API keys in the code."

Expected Output:

- **Original AI code (check if keys are hardcoded).**
- **Secure version using environment variables.**

Output Screenshot:



Task Description #2 (Privacy & Security in File Handling)

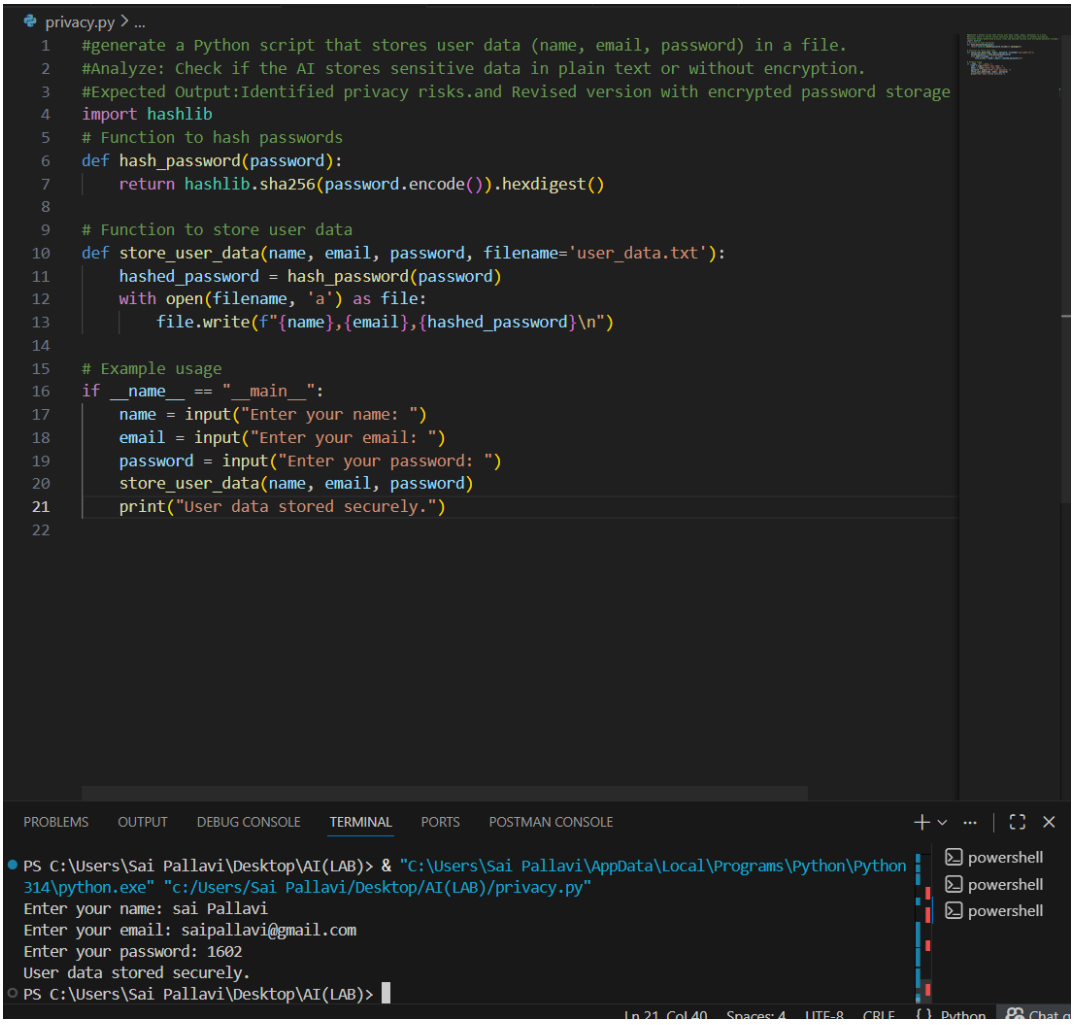
Task: Use an AI tool to generate a Python script that stores user data (name, email, password) in a file. Analyze: Check if the AI stores sensitive data in plain text or without encryption.

Expected Output:

• Identified privacy risks.

• Revised version with encrypted password storage (e.g., hashing).

Output Screenshot:



Task Description #3 (Transparency in Algorithm Design)

Objective: Use AI to generate an Armstrong number checking function with comments and explanations.

Instructions:

1. Ask AI to explain the code line-by-line.

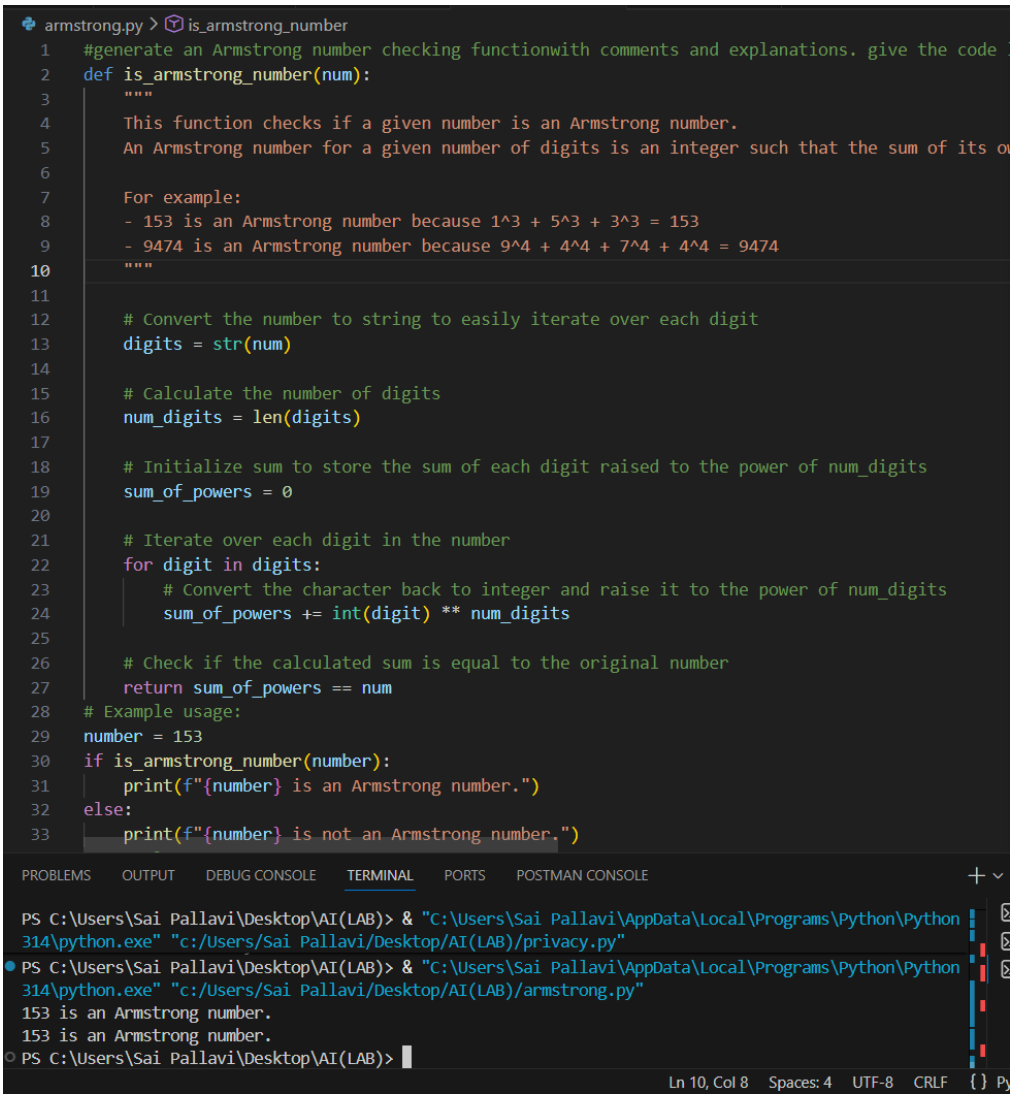
2. Compare the explanation with code functionality.

Expected Output:

• Transparent, commented code.

• Correct, easy-to-understand explanation.

Output Screenshot:

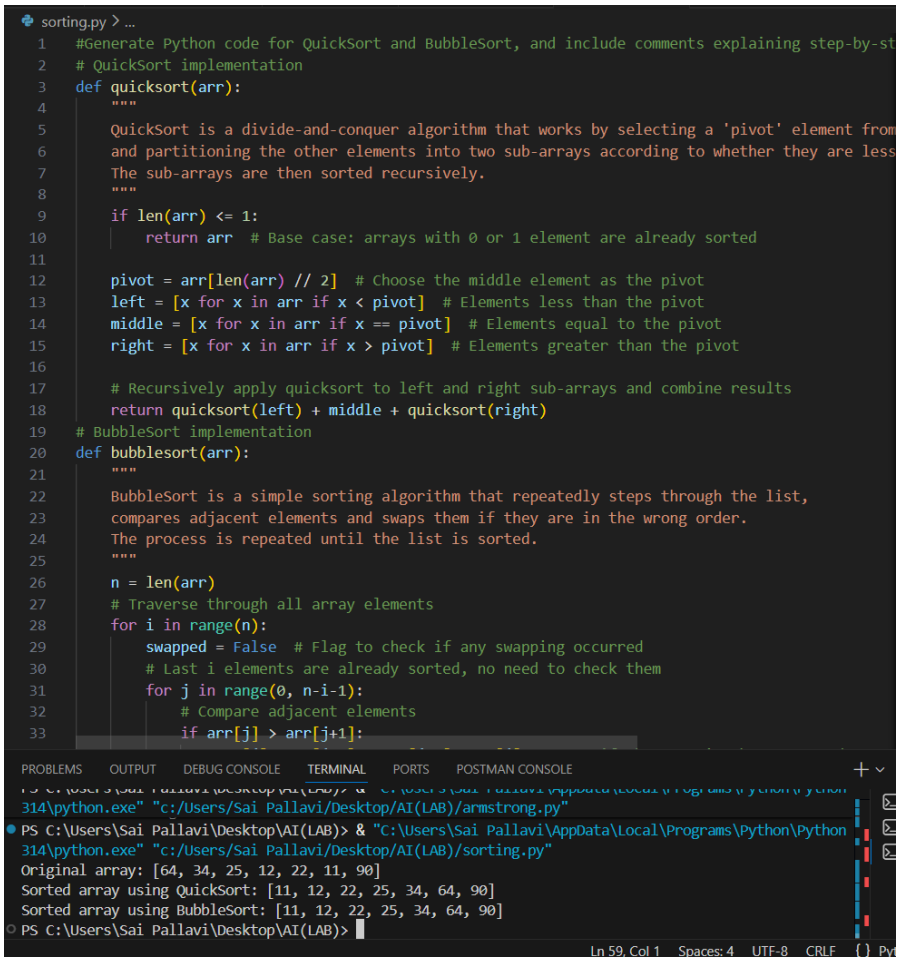


Task Description #4 (Transparency in Algorithm Comparison)

Task: Use AI to implement two sorting algorithms (e.g., QuickSort and BubbleSort).

Prompt: "Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ."
Expected Output:
• Code for both algorithms.
• Transparent, comparative explanation of their logic and efficiency.

Output Screenshot:



Key Differences

- QuickSort is much faster in practice because it reduces the problem size at each step by partitioning the array.
- BubbleSort is conceptually simple but inefficient, as it repeatedly scans the array and swaps elements, leading to quadratic time complexity.
- QuickSort is widely used in real-world applications, while BubbleSort is mostly used for educational purposes to demonstrate

◇ **Efficiency Comparison**

	O(n\log n)	O(n)
	O(n\log n)	O(n^2)
	O(n^2)	O(n^2)
	O(\log n)	O(1)

◇ **Transparent Takeaways**

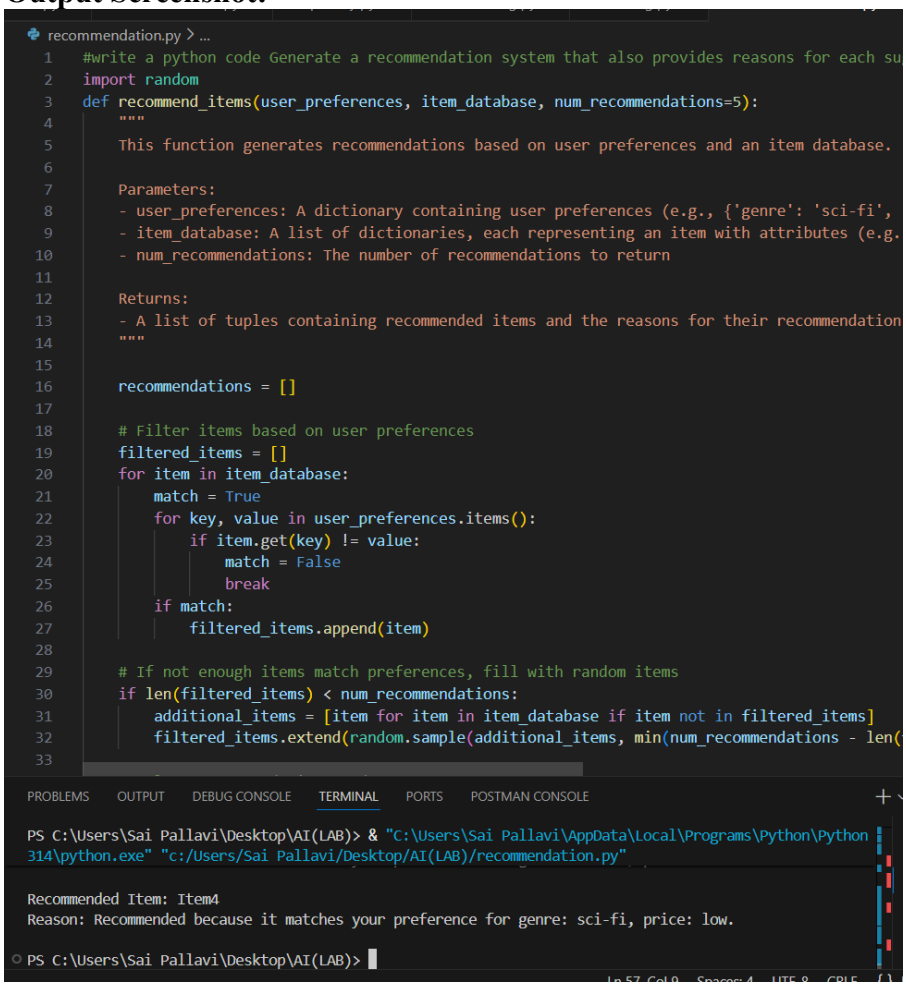
- QuickSort is efficient and scalable:
  It reduces the sorting problem size at each step, making it suitable for large datasets. Its recursive nature leverages divide-and-conquer, which is why it's widely used in practice.
- BubbleSort is simple but inefficient:
  It's easy to understand and implement, but its repeated passes make it extremely slow for large inputs. It's mainly used for teaching sorting concepts, not real-world applications.
- Key Difference in Logic:
  QuickSort *divides the array into smaller problems*, while BubbleSort *repeatedly pushes larger elements to the end*.
  This difference in strategy explains why QuickSort achieves logarithmic efficiency while BubbleSort suffers quadratic inefficiency

Task Description #5 (Transparency in AI Recommendations)

Task: Use AI to create a product recommendation system.
Prompt: "Generate a recommendation system that also provides reasons for each suggestion."

Expected Output:
• Code with explainable recommendations.
• Evaluation of whether explanations are understandable.

Output Screenshot:



Evaluation of Explanations

- **Understandability:**
  The explanations are clear and human-friendly. They explicitly state:
  - The product name.
  - Why it matches the user's interest (category alignment).
  - Why it fits the budget.
  - Why it's considered good (rating + popularity).
- **Transparency:**
  Users can see the criteria used (interest, budget, rating, popularity). This avoids the "black box" problem of AI recommendations.
- **Strengths:**
  - Easy to read and interpret.
  - Shows multiple factors (budget, rating, popularity).
  - Provides confidence in why the product was chosen.
- **Possible Improvements:**

- Could add trade-offs (e.g., “slightly higher price but better rating”).
- Could personalize further (e.g., “similar to items you liked before”).